

Lab 12: Security Hardening End-to-End — TLS Rotation, RBAC, SSO & Audit Pipeline (55 min)

Learning Objectives

By the end of this lab you will be able to:

- Generate a CA, node, and client certificates and start a fully secure cluster
- **Rotate** node and client certificates with zero downtime, and prove the rotation took effect
- Build least-privilege roles with inheritance and default (future) privileges
- Verify privileges with both positive and negative tests, including on newly created tables
- Configure `authentication` methods via `server.host_based_authentication.configuration`
- Wire up an audit pipeline: enable per-table audit, route `SENSITIVE_ACCESS` to its own sink, ship it off-box
- Map CockroachDB features to SOC 2 / GDPR / HIPAA control requirements

Prerequisites

- **Docker Desktop** (or Docker Engine) running — there is no `cockroach` binary to install
- `nc` (netcat) for the log-shipping step
- Lab 9's log-channel configuration is reused here

Setup

Every command in this lab talks to the **secure** cluster, so export this once:

```
export CRDB_COMPOSE=docker/labs-secure.yml
```

(In WSL, `export` as above. Only in native `cmd`: `set CRDB_COMPOSE=docker/labs-secure.yml`.)

Tasks

Part A: Certificates and a Secure Cluster (12 min)

1. **Start it.** The stack generates its own CA, node certificate and root client certificate the first time, then starts a TLS-only node:

```
scripts/crdb up
```

Usage	Certificate File	Key File	Expires	Notes
CA	ca.crt		2036/09/08	num certs: 1
Node	node.crt	node.key	2031/09/05	addresses:
	crdbssl,localhost,127.0.0.1			
Client	client.root.crt	client.root.key	2031/09/05	user: root

2. Read what generated them — the `certs` service in `docker/labs-secure.yml`:

```
docker compose -f docker/labs-secure.yml logs certs
```

It runs the three commands you would run by hand in production:

```
scripts/crdb run cert create-ca --certs-dir=/certs --ca-key=/certs/ca.key
scripts/crdb run cert create-node crdbssl localhost 127.0.0.1 --certs-dir=/certs --ca-key=/certs/ca.key
scripts/crdb run cert create-client root --certs-dir=/certs --ca-key=/certs/ca.key
```

The node certificate lists every name the node will be reached by — here `crdbssl` (inside Docker), `localhost` and `127.0.0.1` (from your machine). Miss one and clients using that name fail TLS verification. In production the list includes the load-balancer name and the service DNS name.

The CA key is the crown jewel. This lab keeps `ca.key` beside the certificates for convenience; in production it lives in a vault or an offline host, and never on a node.

3. Inspect what you have:

```
scripts/crdb run cert list --certs-dir=/certs
```

For the certificate's own details, use a container that has `openssl` (the CockroachDB image does not):

```
docker run --rm -v crdb-labs-secure_crdbssl-certs:/certs postgres:16 \
  openssl x509 -in /certs/node.crt -noout -text | grep -A2 'Validity\|Subject Alternative'
```

4. Connect — certificates required:

```
scripts/crdb sql -e "SELECT 'secure connection OK' AS status;"
```

5. Prove insecure access is rejected:

```
docker compose -f docker/labs-secure.yml exec crdbssl \
  ./cockroach sql --insecure --host=crdbssl -e "SELECT 1;"
```

```
ERROR: node is running secure mode, SSL connection required (SQLSTATE 08P01)
```

Part B: Certificate Rotation Without Downtime (15 min)

Certificates expire. Rotating them under pressure at 2am is how outages happen; rotating them on a schedule is how they don't.

1. Check the current expiry:

```
docker run --rm -v crdb-labs-secure_crdb-certs:/certs postgres:16 \
  openssl x509 -in /certs/node.crt -noout -dates
```

2. Issue a new node cert alongside the old one. `--overwrite` regenerates it signed by the same CA:

```
scripts/crdb run cert create-node \
  crdb1 localhost 127.0.0.1 \
  --certs-dir=/certs \
  --ca-key=/certs/ca.key \
  --overwrite \
  --lifetime=48h
```

List every name again — the reissue replaces the SAN list, it does not extend it. Drop `crdb1` here and the certificate is still perfectly valid, but `scripts/crdb sql` (which connects with `--host=crdb1`) fails TLS verification the moment the node reloads it. Locking yourself out with a *correctly issued* certificate is one of the more memorable ways to end a maintenance window.

3. Signal the node to reload certificates — no restart needed:

```
docker compose -f docker/labs-secure.yml kill -s HUP crdb1
```

⚠ **This only works because the container runs the binary as PID 1.** The CockroachDB image's default entrypoint is a shell script (`cockroach.sh`), and a signal sent to the container goes to *that shell*, which never forwards it — the certificate is reissued on disk, `cert list` shows the new expiry, and the server keeps serving the old certificate. Look for `entrypoint: ["/cockroach/cockroach"]` in `docker/labs-secure.yml`.

It is a containerisation trap worth remembering beyond CockroachDB: **any** process started through a shell wrapper stops receiving signals, so graceful reloads and shutdowns silently stop working.

4. Prove the rotation took:

```
sleep 2
docker run --rm --network crdb-labs-secure_default postgres:16 \
  bash -c
    "echo | openssl s_client -connect crdb1:26257 2>/dev/null | openssl x509 -noout
    -dates"
```

The `notAfter` should now be ~48 hours out. Connections stayed open the whole time:

```
scripts/crdb sql -e "SELECT 'still connected' AS status;"
```

5. **Client cert rotation** follows the same shape — issue, distribute, SIGHUP (or restart the app's connection pool):

```
scripts/crdb run cert create-client root \  
  --certs-dir=/certs --ca-key=/certs/ca.key --overwrite --lifetime=48h  
scripts/crdb sql -e "SELECT 'client rotated' AS status;"
```

6. **The CA rotation problem.** Rotating the CA is harder — every node and client must trust both old and new CAs during the transition. The order is:

1. Append the new CA to ca.crt on every node and client (both trusted)
2. SIGHUP every node
3. Re-issue node and client certs from the new CA, one node at a time
4. Remove the old CA from ca.crt
5. SIGHUP every node again

Doing steps 3 and 4 in the wrong order locks you out of your own cluster. Write it down before you need it.

7. **Rotation policy worth adopting:**

Cert	Lifetime	Rotate at
CA	5–10 years	50% of life, planned
Node	1 year	3 months before expiry
Client	1 year (or shorter for humans)	quarterly, automated

Part C: Least-Privilege RBAC (15 min)

1. **Create the application schema:**

```
scripts/crdb sql <<'SQL'  
CREATE DATABASE ledger;  
USE ledger;  
CREATE TABLE accounts (  
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name    STRING NOT NULL,  
  balance DECIMAL(12,2) NOT NULL,  
  ssn     STRING          -- deliberately sensitive; used in Part E  
);  
INSERT INTO accounts (name, balance, ssn) VALUES  
  ('Alice', 1000, '111-11-1111'), ('Bob', 500, '222-22-2222');  
SQL
```

2. **First, close the default hole.** A new database's `public` schema grants **CREATE** and **USAGE** to the **public** role — meaning *every* user can create objects in it. Check:

```
SHOW GRANTS ON SCHEMA ledger.public;
-- grantee | privilege_type
-- public | CREATE      <-- every user, including one with no grants at all
-- public | USAGE
```

```
REVOKE CREATE ON SCHEMA ledger.public FROM public;
```

This is the single most-missed hardening step. Without it, the careful role hierarchy below is decorative: `app_user` can still `CREATE TABLE` in `public`, because it inherits `CREATE` from the `public` role rather than from any role you granted it. You will verify this with a negative test in step 5 — and it only passes because of this revoke.

3. Build a role hierarchy — roles grant to roles, users get roles:

```
-- Base: can connect and see the schema
CREATE ROLE ledger_connect;
GRANT CONNECT ON DATABASE ledger TO ledger_connect;
GRANT USAGE ON SCHEMA ledger.public TO ledger_connect;

-- Read-only inherits connect
CREATE ROLE ledger_ro;
GRANT ledger_connect TO ledger_ro;
GRANT SELECT ON ALL TABLES IN SCHEMA ledger.public TO ledger_ro;
ALTER DEFAULT PRIVILEGES IN SCHEMA ledger.public GRANT SELECT ON TABLES TO ledger_ro;

-- Read-write inherits read-only
CREATE ROLE ledger_rw;
GRANT ledger_ro TO ledger_rw;
GRANT INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA ledger.public TO ledger_rw;
ALTER DEFAULT PRIVILEGES IN SCHEMA ledger.public
    GRANT INSERT, UPDATE, DELETE ON TABLES TO ledger_rw;

-- Schema owner: can migrate, but is NOT admin
CREATE ROLE ledger_migrator;
GRANT ledger_rw TO ledger_migrator;
GRANT CREATE ON DATABASE ledger TO ledger_migrator;
GRANT CREATE ON SCHEMA ledger.public TO ledger_migrator;
```

4. Create users:

```
CREATE USER report_user WITH PASSWORD 'report-pw'; GRANT ledger_ro TO
report_user;
CREATE USER app_user WITH PASSWORD 'app-pw'; GRANT ledger_rw TO app_user;
CREATE USER migrate_user WITH PASSWORD 'migrate-pw'; GRANT ledger_migrator TO
migrate_user;
```

```
for u in report_user app_user migrate_user; do
    scripts/crdb run cert create-client $u --certs-dir=/certs --ca-key=/certs/ca.key
done
```

5. Inspect the effective grants:

```
SHOW GRANTS ON DATABASE ledger;
SHOW GRANTS FOR app_user;
SHOW ROLES;
```

6. Negative tests — the half people skip:

```
# report_user can read...
scripts/crdb sql --user=report_user -e "SELECT count(*) FROM ledger.public.accounts;"

# ...but must NOT be able to write
scripts/crdb sql --user=report_user -e \
  "UPDATE ledger.public.accounts SET balance = 0;" && echo "PROBLEM" || echo "correctly
  denied"

# app_user can write...
scripts/crdb sql --user=app_user -e \
  "UPDATE ledger.public.accounts SET balance = balance + 1 WHERE name = 'Alice';"

# ...but must NOT be able to create tables
scripts/crdb sql --user=app_user -e "CREATE TABLE ledger.public.bogus (id INT);" \
  && echo "PROBLEM" || echo "correctly denied"

# migrate_user can create
scripts/crdb sql --user=migrate_user -e \
  "CREATE TABLE ledger.public.audit_notes (id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  note STRING);"
```

7. Future privileges — and the part that trips everyone. Try reading the table `migrate_user` just created:

```
scripts/crdb sql --user=report_user -e "SELECT count(*) FROM ledger.public.audit_notes;"
# ERROR: user report_user does not have SELECT privilege on relation audit_notes
```

Why? `ALTER DEFAULT PRIVILEGES` is scoped to **the role that creates the object**. You ran it as `root` in step 3, so it covers tables `root` creates — not tables `migrate_user` creates. Since your migrations run as `migrate_user`, the grants you carefully set up never apply to anything your CI pipeline builds.

It is also scoped to the **current database**: running `ALTER DEFAULT PRIVILEGES IN SCHEMA ledger.public ...` while connected to `defaultdb` silently records nothing useful. Connect to `ledger` first.

Fix it — the migration role declares its own defaults (run this **as** `migrate_user`):

```
scripts/crdb sql --user=migrate_user -d ledger -e "
  ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO ledger_ro;
  ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO ledger_rw;"
```

```
scripts/crdb sql --user=migrate_user -d ledger -e "CREATE TABLE audit_notes_v2 (id UUID
PRIMARY KEY DEFAULT gen_random_uuid(), note STRING);"
scripts/crdb sql --user=report_user -e "SELECT count(*) FROM
ledger.public.audit_notes_v2;" # 0
```

Three ways to configure this, all verified:

Approach	Who runs it	Covers
Role sets its own (above)	<code>migrate_user</code>	Objects that role creates — the CI/CD pattern
<code>ALTER DEFAULT PRIVILEGES FOR</code> <code>ROLE migrate_user ...</code>	admin, must be a member of that role (<code>GRANT migrate_user TO root</code>)	Same, managed centrally
<code>ALTER DEFAULT PRIVILEGES FOR ALL</code> <code>ROLES ...</code>	admin only	Every creator — belt and braces

This is the most valuable RBAC feature for CI/CD *once it's configured for the right role*. Configured for the wrong role, it looks like it works right up until a migration adds a table nobody can read.

8. Revocation is a test too:

```
REVOKE ledger_rw FROM app_user;
```

```
scripts/crdb sql --user=app_user -e "SELECT count(*) FROM ledger.public.accounts;" \
&& echo "PROBLEM" || echo "correctly denied"
```

```
GRANT ledger_rw TO app_user; -- restore for the rest of the lab
```

Part E: The Audit Pipeline (15 min)

1. **Route security channels to their own auditable sink** — write `lab12/logs.yaml` (`mkdir -p lab12` first, so the directory belongs to you rather than to Docker) (the compose stack mounts `./lab12` at `/lab12` inside the node):

```
file-defaults:
  dir: /lab12/logs
sinks:
  file-groups:
    security:
      channels: [SENSITIVE_ACCESS, USER_ADMIN, PRIVILEGES]
      filter: INFO
      auditable: true
      buffered-writes: false # audit records must not be lost on crash
  stderr:
    filter: NONE
```

2. **Restart with the log config** — an overlay adds the flag; the base file is untouched:

```
docker compose -f docker/labs-secure.yml \
  -f docker/labs-secure.logging.yml up -d crdbs1
```

3. Enable per-table audit on the table holding SSNs:

```
ALTER TABLE ledger.public.accounts EXPERIMENTAL_AUDIT SET READ WRITE;
```

4. Generate auditable activity:

```
scripts/crdb sql --user=report_user -e "SELECT name, ssn FROM ledger.public.accounts;"
scripts/crdb sql --user=app_user -e
  "UPDATE ledger.public.accounts SET balance = balance + 5 WHERE name = 'Bob';"
scripts/crdb sql -e "CREATE USER temp_contractor; DROP USER temp_contractor;"
```

5. Read the audit trail:

```
ls lab12/logs/
grep -o '"EventType": "[^"]*"' lab12/logs/cockroach-security*.log | sort | uniq -c
grep 'sensitive_table_access' lab12/logs/cockroach-security*.log | tail -1 | sed 's/
  ^[^{]*//' | python3 -m json.tool
```

The fields that matter downstream:

Field	Why the auditor wants it
Timestamp	When
User	Who
TableName	What data
Statement	What operation (redacted or not, per config)
ApplicationName	Which service
RemoteAddress	From where

6. Ship it off-box. Add an HTTP sink so records leave the node in real time:

```
sinks:
  http-servers:
    siem:
      channels: [SENSITIVE_ACCESS, USER_ADMIN, PRIVILEGES]
      address: http://localhost:9999
      method: POST
      unsafe-tls: true
      buffering:
        max-staleness: 5s
```

```
# Fake SIEM listener in another terminal
while true; do nc -l 9999; echo "--- received ---"; done
```

Restart the node with the updated config, run a `SELECT` against `accounts`, and watch the records arrive.

Why off-box matters: an attacker with node access can delete local log files. Audit records that have already left the machine cannot be un-sent.

7. **Turn the audit off when you're done** (it is not free — it logs every access):

```
ALTER TABLE ledger.public.accounts EXPERIMENTAL_AUDIT SET OFF;
```

Optional — If Time Allows

These parts are not required to complete the lab; they extend it by about 18 minutes. Do them if you finish early, or after class — the cluster and data from the core parts carry over.

Part D: Authentication Methods & SSO (10 min)

1. **See the current HBA configuration:**

```
SHOW CLUSTER SETTING server.host_based_authentication.configuration;
```

An empty value means the defaults are in force: cert or password for everyone.

2. **Require certificates for the app, allow passwords for humans:**

```
SET CLUSTER SETTING server.host_based_authentication.configuration = '  
# TYPE  DATABASE  USER          ADDRESS      METHOD  
host    all       app_user      all          cert  
host    all       report_user   all          cert-password  
host    all       root         all          cert  
host    all       all          all          scram-sha-256  
';
```

3. **Verify the rules are in force:**

```
SHOW CLUSTER SETTING server.host_based_authentication.configuration;
```

⚠ `pg_hba_file_rules` is empty in CockroachDB. It exists for PostgreSQL tool compatibility, but CRDB's HBA configuration lives in the cluster setting above, not in a `pg_hba.conf` file — so querying that view tells you nothing about what is actually enforced. Read the setting back instead, and prove the rules with a real connection attempt (step 6).

4. **Confirm password hashing uses SCRAM:**

```
SHOW CLUSTER SETTING server.user_login.password_encryption;
```

scram-sha-256 should be the value. crdb-bcrypt is the legacy setting — if you see it, plan a migration.

5. SSO / OIDC — the configuration shape (needs a real IdP, so this is a walkthrough):

```
-- DB Console SSO
SET CLUSTER SETTING server.oidc_authentication.enabled = true;
SET CLUSTER SETTING server.oidc_authentication.provider_url = 'https://idp.example.com';
SET CLUSTER SETTING server.oidc_authentication.client_id = '<client-id>';
SET CLUSTER SETTING server.oidc_authentication.client_secret = '<secret>';
SET CLUSTER SETTING server.oidc_authentication.redirect_url = 'https://crdb.example.com/oidc/v1/callback';
SET CLUSTER SETTING server.oidc_authentication.claim_json_key = 'email';
SET CLUSTER SETTING server.oidc_authentication.principal_regex = '^(^@+).example\.com$';

-- Cluster SSO for SQL clients (JWT)
SET CLUSTER SETTING server.jwt_authentication.enabled = true;
SET CLUSTER SETTING server.jwt_authentication.issuers = 'https://idp.example.com';
SET CLUSTER SETTING server.jwt_authentication.audience = 'crdb-cluster';
```

```
-- Roll them back so the rest of the lab keeps working
SET CLUSTER SETTING server.oidc_authentication.enabled = false;
SET CLUSTER SETTING server.jwt_authentication.enabled = false;
```

The operational point: with SSO, deprovisioning a human in the IdP removes their database access. Without it, offboarding depends on someone remembering to run `DROP USER`.

6. Lock down accounts that should never log in interactively:

```
ALTER USER app_user NOLOGIN; -- then check
```

```
scripts/crdb sql --user=app_user -e "SELECT 1;" && echo "PROBLEM" || echo "correctly denied"
```

```
ALTER USER app_user LOGIN;
```

Part F: Compliance Mapping (8 min)

Fill in which CockroachDB feature satisfies each control, and what evidence you'd hand an auditor.

Control requirement	Framework	CockroachDB feature	Evidence to produce
Encryption in transit	SOC 2, HIPAA		
Encryption at rest	SOC 2, HIPAA		
Access control / least privilege	SOC 2, GDPR		

Control requirement	Framework	CockroachDB feature	Evidence to produce
Audit trail of data access	SOC 2, HIPAA		
Right to erasure	GDPR		
Data residency	GDPR		
Backup and recoverability	SOC 2		
Deprovisioning of access	SOC 2		

Hints: `--certs-dir` / TLS; enterprise encryption-at-rest (`--enterprise-encryption`); roles + default privileges; `SENSITIVE_ACCESS` channel; row deletion + TTL + backup expiry; `REGIONAL BY ROW` with region-pinned zone configs (Lab 7); scheduled backups and a measured DR drill (Lab 11); SSO/OIDC.

Cleanup

```
docker compose -f docker/labs-secure.yml down -v
rm -rf lab12/logs
```

That removes the node, its data **and** the certificate volume, so the next run generates a fresh CA.

Lab 12 Deliverables

□ **Secure cluster** with CA/node/client certs; insecure connection proven to fail □ **Zero-downtime rotation** of node and client certs, verified on the wire with `openssl s_client` □ **CA rotation procedure** written down in the correct order □ **public schema CREATE revoked** — the default that silently defeats least privilege □ **Role hierarchy** with inheritance and default privileges, verified positive *and* negative □ **Future privileges configured for the migration role** — not just for `root` □ **HBA configuration** requiring certs for service accounts and SCRAM for humans □ **Audit pipeline** with a dedicated auditable sink and off-box shipping □ **Compliance map** from control requirement to feature to evidence

Challenge Exercises

1. **Rotate the CA end to end** on a 3-node cluster without dropping a connection. Script it, then have a colleague run your script without your help.
2. **Alert on privilege escalation.** Write a check that greps the `USER_ADMIN` channel for `GRANT ... TO ...` involving the `admin` role and fires a webhook. What is the acceptable detection latency for that alert?

3. **Column-level protection.** `ssn` should not be readable by `report_user`. Solve it two ways — a view with column-level grants, and revoking `SELECT` on the column (`GRANT SELECT (id, name, balance) ON ...`). Which one survives a schema migration better?
4. **Redaction in logs.** Enable `sql.log.redact_names.enabled` / run with `--redact` and compare what an auditor can still learn from the audit trail. What did you lose?

Reference

Command	Purpose
<code>scripts/crdb run cert create-ca/node/client</code>	Issue certificates
<code>cockroach cert list --certs-dir=...</code>	Inspect issued certs and expiry
<code>docker compose kill -s HUP <svc></code>	Reload certs without restarting the node (needs the binary as PID 1)
<code>openssl s_client -connect host:port</code> (sidecar container)	Verify the cert actually served
<code>GRANT <role> TO <role></code>	Role inheritance
<code>REVOKE CREATE ON SCHEMA x.public FROM public</code>	Close the default "anyone can create" grant
<code>ALTER DEFAULT PRIVILEGES ...</code>	Future objects — scoped to the creating role and the current database
<code>ALTER DEFAULT PRIVILEGES FOR ALL ROLES ...</code>	Future objects from any creator (admin only)
<code>server.host_based_authentication.configuration</code>	Per-user/host auth methods
<code>server.oidc_authentication.*</code>	DB Console SSO
<code>server.jwt_authentication.*</code>	Cluster SSO for SQL clients
<code>ALTER TABLE ... EXPERIMENTAL_AUDIT SET READ WRITE</code>	Per-table audit logging
<code>--log-config-file</code> with <code>auditable: true</code>	Tamper-evident audit sink